EVOLVE

구조를 진화시키는 법

변경 압력의 분포, QueryEngine 추출, Tool 분해, feature variability와 운영 피드백

> **이 코드베이스의 구조 문제는 큰 파일 자체보다 제품 변형과 effect 정합성이 여러 entrypoint에 중복된 데서 나온다.**

대상
v2.1.88 / 2ca5ddabfed5

관점
refactoring · variability · operations

제3자 복원 소스에 대한 교육용 분석이다. 누락된 모듈과 복원 한계를 구현 사실로 보완하지 않는다.

현재 구조는 “완성된 계층”보다 진화 중인 seams로 읽어야 한다

복원본에는 새로운 단일 source of truth를 만들려는 주석과 아직 남은 중복 경로가 동시에 보인다.

추출

QueryEngine

ask의 conversation lifecycle을 class로 옮겼으나 REPL은 아직 query를 직접 사용.

통합

assembleToolPool

REPL과 agent worker의 built-in/MCP 조립을 한 함수로 모음.

중앙화

buildTool

도구 기본값과 타입 추론을 한 곳으로 이동.

격리

query config/deps

immutable snapshot과 I/O dependency를 loop에서 분리.

읽는 법	파일 배치를 이상적 architecture로 평가하기보다, 어떤 중복과 race를 없애기 위해 새 seam이 생겼는지 본다.

실행 경로는 하나지만 소유권은 여러 entrypoint에 퍼져 있다

interactive, print/SDK, bridge, subagent가 공통 query와 tools를 사용하면서도 lifecycle 조립은 각자 가진다.

surface	공유하는 core	자체 소유하는 것
REPL	query, tools, message helpers	React store, queryGuard, prompt restore, Ink projection
print/SDK	QueryEngine→query, tools	structured IO, batching, SDK result, process lifecycle
bridge/remote	message/SDK protocol	transport, reconnection, remote input policy
subagent/task	query, tool runtime	forked context, sidechain transcript, parent progress

공통 core가 있다는 사실과 공통 owner가 있다는 사실은 다르다. 같은 helper를 호출해도 state initialization·flush·cancel 순서가 다르면 제품 semantics가 갈라진다.

큰 파일은 원인이 아니라 여러 변화 이유가 만나는 장소다

상위 파일 크기는 responsibility fan-in을 찾는 출발점으로만 사용한다.

파일	약 LOC	겹친 변화 이유
cli/print.ts	5.6k	SDK IO·permission·queue·MCP·resume·process
utils/messages.ts	5.5k	protocol normalization·factories·selection·repair
utils/sessionStorage.ts	5.1k	append queue·metadata·resume·compact·search
utils/hooks.ts	5.0k	hook discovery·execution·events·policy
screens/REPL.tsx	5.0k	UI·turn lifecycle·queue·telemetry·cancel
main.tsx	4.7k	CLI composition·mode routing·startup
services/api/claude.ts	3.4k	provider request·cache·stream·retry·telemetry

리팩터링 오해

라인 수를 줄이는 분할은 import 수만 늘릴 수 있다. 먼저 각 파일의 state owner, effect point, compatibility policy, projection consumer를 표시해야 한다.

변경 압력은 서로 다른 영역에 다르게 쌓였다

주석의 reliability/performance/security/concurrency 어휘를 탐색 지표로 보면 각 slice의 주된 부담이 다르다.

slice	두드러진 압력	구조적 의미
conversation engine	recovery·cache·fallback	transition policy와 transport가 결합
context/persistence	performance·resume·stale	projection과 journal이 결합
shell effects	security·compatibility	parser/policy/runtime가 한 capability에 집중
interactive/SDK	reliability·concurrency	UI가 lifecycle coordination까지 소유
extension adapters	performance·reliability	외부 공급자 수명과 core turn이 결합

해석 주의

lexical count는 defect나 품질 점수가 아니다. refactoring discovery surface를 좁히는 데 쓰며, 실제 판단은 해당 state/effect 경계를 읽고 내린다.

가장 중요한 구조 축은 REPL과 QueryEngine의 합류다

QueryEngine은 headless/SDK conversation owner이고, REPL은 직접 query 호출과 자체 state coordination을 유지한다.

진행 중인 구조 seam

```
today:
REPL lifecycle ──────────────────────┐
                                     ├→ query() → tools
print/SDK → QueryEngine ─────────────┘

direction stated in source:
REPL adapter ──┐
SDK adapter  ──┼→ QueryEngine/ConversationEngine → query/tools
bridge       ──┘
```

이 migration의 성공 기준은 코드 재사용률이 아니다. 동일 prompt와 tool fixture에서 event ordering, permission, transcript, terminal result가 entrypoint 간 동등해야 한다.

코드 좌표 src/QueryEngine.ts · src/screens/REPL.tsx

Tool 계약은 강한 응집과 shotgun change를 동시에 만든다

새 tool을 추가하면 model schema부터 UI까지 한 정의 안에서 볼 수 있지만, headless build도 React presentation에 닿는다.

현재 장점	현재 비용	분해 시 지킬 것
한 정의로 누락 방지	서로 다른 consumer의 dependency 결합	identity/schema 단일 소유
tool별 응집	UI-only 변경이 runtime 타입에 영향	projection adapter 분리
default 중앙화	default semantics가 모든 tool에 적합하다는 착시	capability별 명시적 override
입력별 policy	permission/concurrency classifier 중복 parse 가능	validated input 공유

코드 좌표 src/Tool.ts

feature gate는 조건문이 아니라 product-line architecture다

AST 기준 feature() 호출이 광범위하며, 일부 모듈은 conditional require로 build output에서 문자열까지 제거한다.

이 환경에서는 “flag off면 실행 안 됨”만으로 부족하다. bun:bundle의 tree-shaking 제약 때문에 feature literal이 어떤 조건식에 등장하는지, gated module을 static import하는지까지 구조에 영향을 준다.

변형 축	예	구조 영향
제품 build	external/ant/simple/cowork	도구·prompt·telemetry·문자열 포함 여부
runtime mode	REPL/SDK/agent/plan	state owner와 permission
provider/model	1P/Bedrock/Vertex/model capability	beta header·tool search·thinking
experiment	compact/stream execution/agent feature	recovery path와 persistent format

코드 좌표 src/query.ts · src/services/api/claude.ts

dead-code elimination 요구가 module boundary를 만든다

feature-gated string과 import가 외부 build에 남으면 안 되는 경우 dynamic require/import가 사용된다.

build-time variability pattern

```
const optional = feature("X")
  ? require("./feature-module.js")
  : null

// call sites must remain inside recognized feature condition
```

일반적인 dependency injection 관점에서는 어색해 보여도, 여기서는 artifact 내용 자체가 acceptance constraint다. 리팩터링 시 static import로 바꾸면 runtime은 같아도 build가 다른 제품이 된다.

> **배경지식**
> 이것은 Strategy pattern보다 **software product line**과 conditional compilation 문제다. architecture fitness에는 동작 테스트뿐 아니라 excluded symbol/string 검사도 필요하다.

주석에서 개발 운영의 흔적을 읽는다

얕은 clone이라 Git history는 사용할 수 없지만, 코드에는 사고·성능 측정·compatibility 회귀를 막기 위한 결정 기록이 풍부하다.

주석 패턴	드러나는 작업	보고서 해석
Without this…	특정 failure chain의 원인과 결과	불변조건 후보
Previously…	이전 설계와 회귀 방지	migration seam
inc-/GH/#…	운영 사건·외부 issue와 연결	실제 pressure signal
ms/MB/token 수치	profiling·production observation	성능 판단 근거
TODO: make required/extract	의도된 contract tightening	refactoring 후보

주의

주석은 유용하지만 최신성·완전성을 보장하지 않는다. 동작 코드와 관측 지표를 함께 읽고, 조직의 실제 프로세스나 원인을 과잉 추정하지 않는다.

telemetry는 리팩터링의 동등성 oracle이 될 수 있다

logEvent와 queryCheckpoint가 광범위하게 분포한다. 단순 usage analytics가 아니라 failure funnel과 critical path를 설명하는 event도 많다.

chain

queryChainId/depth

재귀 turn과 subquery를 한 인과 흐름으로 연결.

request

request id/model/provider

stream/fallback/retry를 같은 시도로 결합.

effect

tool name/id/duration/result

권한·실행·오류 단계 비교.

durability

resume consistency

in-session과 restored projection 차이 감시.

운영과 설계의 연결	새 architecture로 옮기기 전에 현 event trace를 golden baseline으로 삼으면, source 구조가 바뀌어도 product semantics의 손실을 탐지할 수 있다.

호환성은 경계마다 다른 형태로 나타난다

legacy alias 하나만의 문제가 아니다. protocol, serialized transcript, provider, OS, Node version, tool schema가 모두 compatibility surface다.

surface	예	격리 방향
tool identity	deprecated alias	registry resolver
observable input	legacy/derived fields	observer adapter clone
transcript	old message/metadata shape	versioned reader/projection
provider	beta header/tool-search support	provider capability adapter
OS/shell	Windows path/PowerShell/mtime	platform effect adapter
runtime	Node 18 API 차이	compat utilities/build target

호환 코드를 core branch 곳곳에 두면 새 기능과 곱해진다. “compatibility normalization before core”와 “projection after core” 두 방향으로 몰아야 한다.

리팩터링 우선순위를 결합도와 오류 비용으로 정한다

큰 파일 순서가 아니라 state ownership과 effect 위험으로 정렬한다.

후보	가치	위험	우선 조건
REPL→QueryEngine 합류	turn semantics 단일 owner	interactive race/latency 회귀	golden event trace
Tool facet 분해	headless/UI dependency 절단	schema/prompt drift	adapter + contract fixture
session journal/projection 분리	resume reasoning 단순화	format 호환·data loss	versioned replay corpus
recovery policy 추출	query branch 축소·반복 guard 명시	event 순서 변화	failure matrix
context compiler IR	cache/provenance 명시	cache hit 저하	request hash baseline

1단계: 현재 의미를 trace로 고정한다

코드 이동 전에 한 turn의 사건과 외부 효과를 기록하는 characterization harness가 필요하다.

1. fixture: text-only success, Read→Edit→Bash, permission deny, user abort, stream fallback, prompt-too-long, resume.
2. 관측: CoreEvent sequence, tool ids, terminal reason, provider request hash, transcript projection, filesystem diff.
3. entrypoint: REPL adapter와 SDK adapter가 같은 fixture에서 허용된 차이만 보이는지 비교.
4. feature: 주요 gate on/off 조합을 pairwise로 선택하고 excluded symbol 검사 포함.

> **왜 먼저인가**
>
> 복원 저장소에는 완전한 test suite와 build module이 없다. 새 unit test만 쓰면 현재 암묵 contract를 놓치므로 characterization이 선행되어야 한다.

2단계: ConversationEngine의 ownership을 완성한다

QueryEngine을 단순 wrapper가 아니라 turn state·durability boundary의 owner로 만든다.

01	protocol	UI/SDK 독립 CoreInput/CoreEvent/Terminal 정의
02	engine	context build→query→journal→settlement 순서 소유
03	policy ports	permission, recovery, durability mode를 주입
04	adapters	REPL/SDK/bridge가 projection과 interaction만 소유

REPL을 한 번에 옮기지 말고 shadow execution 또는 trace comparison으로 단계화한다. queryGuard와 prompt restore처럼 interactive-only인 책임은 adapter에 남긴다.

3단계: Tool과 session의 변화 이유를 분리한다

engine 통합 뒤에 effect와 durability를 각각 좁힌다.

작업	먼저 만드는 seam	acceptance
Tool 분해	기존 Tool→facet adapter	모든 기존 tool schema/result snapshot 동일
presentation 분리	ToolPresenter registry	REPL render와 transcript search fidelity
journal 분리	append writer + immutable records	crash/flush ordering 유지
resume 분리	versioned projector/repair stages	historical corpus 재생
recovery 추출	transition table + guard state	동일 failure trace/호출 횟수

순서의 이유

동시에 바꾸면 어떤 layer가 semantic drift를 만들었는지 찾기 어렵다. engine owner를 먼저 고정하고 effect/projection/durability를 한 축씩 옮긴다.

새 구조는 자동화된 경계 검사로 유지한다

문서상의 목표만으로는 다시 결합된다.

fitness rule	막는 회귀
engine core는 React/Ink import 금지	headless core가 presentation에 재결합
provider request는 ContextIR assembler만 생성	cache prefix 규칙 분산
ToolEffect는 Presenter import 금지	실행과 UI 결합
journal record는 versioned union	silent resume format drift
feature-gated module static import 검사	build artifact에 excluded code 유입
CoreEvent exhaustive projection	새 event가 REPL/SDK 중 한쪽에서 누락

구조의 완료 조건

디렉터리가 예뻐지는 것이 아니라, 잘못된 dependency와 누락된 variant가 CI에서 실패해야 한다.

개발 운영은 세 종류의 feedback loop로 닫힌다

설계·성능·안전 판단을 배포 후 증거와 연결한다.

01	correctness loop	failure trace→불변조건→characterization→refactor
02	performance loop	checkpoint/production distribution→critical path 변경→회귀 budget
03	rollout loop	feature gate→cohort telemetry→kill switch→cleanup/deletion

feature flag를 영구 architecture로 남기지 않으려면 각 flag에 owner, 비교 지표, 제거 조건, persistent format 영향이 필요하다. 그렇지 않으면 recovery branch와 provider 조합이 계속 곱해진다.

이 프로젝트를 공부했다는 증거

파일을 많이 읽었는지가 아니라 설계 판단을 반례와 함께 설명할 수 있는지를 본다.

수준	설명할 수 있어야 하는 것	실습 산출물
구현	Edit가 stale write를 어떻게 막는가	read/edit race 재현
통합	tool result가 다음 model request와 transcript에 어떻게 합류하는가	end-to-end trace
실패	stream fallback/abort에서 orphan과 double effect를 어떻게 막는가	failure matrix
구조	REPL/SDK 중복 ownership과 migration seam	responsibility map
운영	cache/latency/security 지표가 코드 구조를 어떻게 바꾸는가	metric-linked ADR
리팩터링	단계별 oracle과 rollback 기준	phased change proposal

> **발표 기준**
> 상위 흐름을 나열하는 대신 하나의 불변조건을 고르고, 정상·실패·resume 경로에서 어떤 코드와 state가 그것을 보존하는지 설명한다.

이 분석이 말할 수 없는 것

복원본의 구조만으로 실제 조직·테스트·배포 체계를 완전히 알 수 없다.

- shallow clone은 장기 Git history와 실제 refactor chronology를 제공하지 않는다.
- README가 밝힌 누락 feature modules 때문에 import graph와 build 결과가 불완전하다.
- feature() 호출 존재는 현재 사용자 build에서 활성임을 뜻하지 않는다.
- 주석의 incident 번호와 production 수치는 외부 record 없이 독립 검증할 수 없다.
- 제안한 target architecture는 교육적 재구성이며 upstream roadmap이 아니다.

이 한계를 유지하면서도 state owner, effect boundary, protocol invariant, 현재 코드의 explicit migration comment는 충분히 학습할 수 있다.

구조 개선의 중심 원칙

이 프로젝트는 복잡해서 계층을 더 만드는 것이 아니라, 외부 효과와 장기 상태를 안전하게 다뤄야 해서 복잡하다.

- turn semantics를 한 conversation owner로 모은다.
- model representation, authority, effect, projection, durability를 분리한다.
- cache·resume·compatibility처럼 bytes와 history에 민감한 계약은 먼저 oracle을 만든다.
- feature variability를 build/runtime/provider 축으로 명시하고 조합 폭발을 관리한다.
- 운영 telemetry를 새 구조의 동등성·성능·안전 acceptance에 사용한다.
- 리팩터링은 파일 이동이 아니라 불변조건의 소유권을 선명하게 만드는 작업이다.